

PRISM-Loc: Three LiDAR Localization Backends Behind One ROS 2 Contract, with Middleware-Free Estimator Cores

Jungmo Kang¹

¹Independent Researcher, Seoul, Republic of Korea

Map-based LiDAR localization for mobile robots is solved in fragments: a 2D scanner calls for an AMCL-style particle filter over an occupancy grid, a 3D LiDAR for an NDT-based tracker over a point-cloud map, and an inertial- or GNSS-aided platform for a Kalman fusion node, each with its own topics, parameters, initialization ritual, and recovery behavior. We introduce *PRISM-Loc*, an open-source ROS 2 localization stack that places three established estimator families behind one node contract: *laser2d*, a likelihood-field Monte Carlo localizer; *ndt3d*, an NDT-MCL backend that reuses the same particle filter with a Gaussian-voxel measurement model; and *fusion3d*, a 15-state error-state Kalman filter fusing LiDAR, IMU, and RTK-GNSS. Every backend publishes the same map→odom transform and pose with covariance and accepts the same `/initialpose` seeding, all filter mathematics live in Eigen-only C++17 cores that are tested without a ROS runtime, and *laser2d* relocalizes a kidnapped robot on request (at startup or by a service call) by deterministic branch-and-bound scan matching over a map-sized search window restricted to cells not marked occupied. We claim no new estimator theory; the contribution is architectural. In seeded closed-loop experiments on *synthetic* data that drive the actual cores, the 2D tracker holds a steady-state mean position error of 0.105 m, branch-and-bound relocalization succeeds on 97 of 100 kidnapped-robot queries in a 6 m room with zero false accepts at its default threshold, but on 40 m and 80 m maps with repeated structure it still accepts a wrong pose on 6 and 15 of 120 queries, and the ESKF bounds a 10 s aiding dropout to 0.125 m RMSE, although a 100-run NEES test shows that its covariance is overconfident (full-state NEES above the 95% interval at 89 of 120 epochs, mainly in attitude; bias blocks consistent); no field-accuracy claim is made. Code: https://github.com/kjungmo/prism_loc.

Keywords: mobile robots, LiDAR localization, Monte Carlo localization, NDT, error-state Kalman filter, ROS 2

1. Introduction

Localizing a mobile robot against a prior map is, on paper, a solved problem, but in practice it is solved in fragments. A team fielding a 2D LiDAR reaches for `nav2_amcl`, a particle filter that tracks a planar pose against an occupancy grid and nothing else. A team with a 3D LiDAR reaches instead for an `hdl_localization`-style tracker built around 3D NDT registration (Koide et al., 2019). A team that must fuse inertial and satellite measurements reaches for a `robot_localization`-style node (Moore and Stouch, 2016), which composes an EKF over odometry, IMU, and GNSS but treats the map-relative LiDAR measurement as someone else’s problem. Each of these tools carries its own topic interface, parameter vocabulary, initialization ritual, and recovery story, so a practitioner who moves from a 2D platform to a 3D one, or who adds a GNSS receiver, re-integrates from scratch.

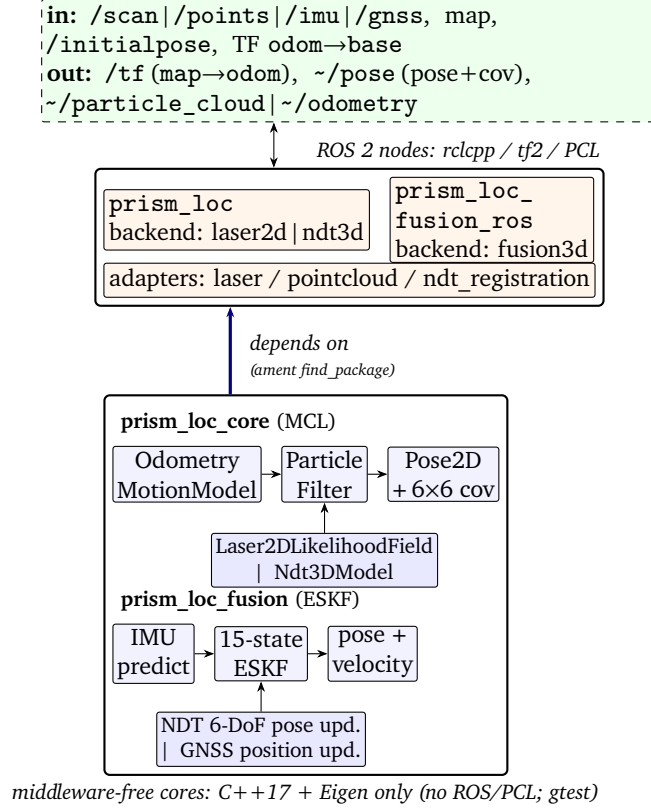


Figure 1: PRISM-Loc overview. Two middleware-free estimator cores (bottom, C++17 + Eigen only): `prism_loc_core` runs the MCL loop (ODOMETRYMOTIONMODEL → PARTICLEFILTER, corrected by either the LASER2DLIKELIHOODFIELD or the NDT3DMODEL measurement model, plus a branch-and-bound relocalizer), and `prism_loc_fusion` runs a 15-state ESKF (IMU prediction, corrected by NDT 6-DoF pose and GNSS position updates). The ROS 2 node packages (top) consume the cores upward through `ament`, select a backend at launch, bridge messages through adapters, and expose one shared contract: a `map→odom` TF, a pose-with-covariance output, and an `/initialpose` seed input.

The gap is recognized by the maintainers of the ROS 2 navigation ecosystem. Their survey (Macenski et al., 2023) catalogs the shortcomings of AMCL and sketches, as unbuilt future work, a modular framework of swappable 2D and 3D localization components behind a common interface. The cost of the gap is not merely inconvenience: the Nav2 marathon deployment (Macenski et al., 2020) names low localization confidence as one of the two causes behind most of its recovery triggers, so how a stack initializes, degrades, and recovers is a first-class deployment concern rather than a detail. Classical MCL recovers from such failures through stochastic mechanisms, random particle injection or dual sampling (Fox et al., 1999; Thrun et al., 2001), which are hard to reproduce in a test.

To overcome this fragmentation, we introduce *PRISM-Loc*, an engineering response that packages three map-based LiDAR localization backends behind one ROS 2 node contract (Figure 1): `laser2d`, a 2D likelihood-field Monte Carlo localizer against an occupancy grid; `ndt3d`, a 3D NDT-MCL backend that reuses the same particle filter with a voxel-Gaussian measurement model over a prior point cloud, sampling the planar (x, y, θ) pose; and `fusion3d`, a 15-state error-state Kalman filter (ESKF) that fuses 3D LiDAR, IMU, and RTK-GNSS with IMU-bias estimation. All three present the identical interface, a `map→odom` transform, a pose with 6×6 covariance, and `/initialpose` seeding, so choosing between a 2D scan, a 3D cloud, and an inertial/GNSS-aided configuration is a launch-time decision rather than a change of stack. We claim no new estimator theory: likelihood-field MCL, NDT-MCL, ESKF fusion, and branch-and-bound scan matching are all established. The contribution is

the factoring, and it is delivered as usable open-source software:

- *One Pluggable Contract*: three estimator families (2D likelihood-field MCL, 3D NDT-MCL, and LiDAR+IMU+RTK-GNSS ESKF fusion) present one ROS 2 map→odom contract, so the sensor suite selects a backend rather than a separate stack; the release adds fail-fast configuration checks, a startup watchdog that names the input that has gone silent, and a single-source parameter reference.
- *Middleware-Free, Deterministic Cores*: all filter mathematics sit behind an Eigen-only boundary with no `rclcpp`, `tf2`, or `PCL`, and all sampling routes through one seeded generator, so every algorithmic claim is exercised by 52 deterministic unit and integration tests and by closed-loop synthetic experiments that compile the cores directly.
- *Deterministic Global Relocalization*: for `laser2d`, branch-and-bound correlative scan matching over a max-pooled likelihood pyramid recovers a kidnapped robot from one scan and re-seeds the MCL posterior, replacing stochastic random-injection recovery with a repeatable search that is provably optimal for its correlative score over the non-occupied (free or unknown) cells of the rotation–translation lattice; the search window is sized from the map, so the search is global up to a configurable compute cap (100 m per axis by default), and the acceptance test, not the search, is what limits it on large maps with repeated structure.

We validate PRISM-Loc at the level of correctness and closed-loop behavior, not field accuracy. On seeded synthetic worlds generated to match the filters’ assumptions, the `laser2d` tracker holds a steady-state mean position error of 0.105 m while KLD-adaptive resampling shrinks the particle set from the 2000-particle ceiling to the 500-particle floor; branch-and-bound relocalization recovers 97 of 100 kidnapped-robot queries in a 6 m room under up to 80% beam clutter with zero false accepts at its default acceptance threshold, and on 40 m and 80 m maps recovers 50 and 41 of 60 poses that lie outside the fixed window of our first release, which recovered 2 and 0; there, however, it still accepts a wrong pose on 6 and 15 of 120 queries because repeated shelves look alike; and the ESKF limits a 10 s loss of all aiding to 0.125 m position RMSE, while a 100-run Monte Carlo NEES test shows that its covariance is overconfident, mainly in attitude. These numbers demonstrate that the assembled pipelines behave as intended; they are not a benchmark, and quantitative evaluation on public datasets remains future work (Section 5).

2. Related Work

Monte Carlo Localization. Particle-filter localization was introduced by Dellaert et al. (1999) and Fox et al. (1999), who represent the pose posterior as a weighted sample set updated by motion-model prediction, sensor-likelihood weighting, and resampling, and who demonstrated global localization from a uniform prior on real robots. Thrun et al. (2001) later hardened MCL against accurate sensors and the kidnapped-robot problem via Mixture-MCL’s dual sampler, and Fox (2003) made the sample size adaptive through KLD-sampling; the odometry motion model and likelihood-field sensor model we use are the textbook forms of Thrun et al. (2005). The particle-filter core of PRISM-Loc is a direct descendant of this line and of its `nav2_amcl` lineage; we contribute no new filter theory. What differs is factoring: the same predict–weight–resample core is shared, unchanged, between a 2D likelihood-field backend and a 3D NDT backend, and recovery is delegated to a deterministic branch-and-bound search (Section 3.4) rather than stochastic injection or dual sampling.

NDT Representations and Measurement Models. Biber and Straßer (2003) introduced the Normal Distributions Transform, modeling a scan as per-cell Gaussians and reducing registration to Newton optimization of a smooth score; Magnusson et al. (2007) generalized the representation to 3D voxel

grids for field deployment on mining vehicles. Both use NDT for deterministic scan-to-scan or scan-to-map registration that requires a reasonable initial guess. The `ndt3d` backend of PRISM-Loc adopts the Gaussian-voxel representation but reads the NDT score out as a per-particle likelihood inside the shared MCL core (NDT-MCL rather than NDT registration), trading the quadratic convergence of Newton alignment for the multi-hypothesis robustness of the particle filter, so the 2D and 3D backends differ only in their measurement model.

LiDAR-Inertial Fusion and Robot Software Systems. Solà (2017) is the mathematical basis of the `fusion3d` backend: the nominal/error-state decomposition, local angular-error parameterization, and predict–correct–inject–reset cycle are implemented essentially as derived there. On the systems side, Moore and Stouch (2016) established the configurable EKF/UKF fusion node for ROS, Wan et al. (2018) demonstrated an industrial ESKF fusing GNSS-RTK, LiDAR, and IMU at centimeter accuracy as a production system tied to its own pre-built LiDAR maps, and Nav2 (Macenski et al., 2020) treats localization as an external, swappable component. Tightly-coupled LiDAR-inertial odometry such as FAST-LIO (Xu and Zhang, 2021), and applied 3D map-based tracking systems (Koide et al., 2019), are complementary: they produce ego-motion and maps, whereas PRISM-Loc localizes within a prior map. Relative to these, PRISM-Loc places three known estimator families behind one ROS 2 topic and TF contract, each implemented as a library that is unit-testable without a middleware runtime, a property that `robot_localization`’s otherwise modular design did not prioritize, and which answers the modular-localization framework that Macenski et al. (2023) left as future work.

Global (Kidnapped-Robot) Localization. Olson (2009) formulated 2D scan matching as exhaustive correlative search over a rasterized log-likelihood table with coarse-to-fine multi-resolution pruning, and Hess et al. (2016) bounded that search with branch-and-bound over a precomputed grid pyramid to run loop closure in real time inside Cartographer. PRISM-Loc borrows exactly this branch-and-bound-over-pyramid primitive but applies it in a narrower setting: one-shot relocalization against a known static map, with no submaps or pose graph to maintain, whose result re-seeds the particle filter in place of the heuristic recovery of classical MCL (Fox et al., 1999; Thrun et al., 2001).

3. PRISM-Loc: One Localization Contract over Three Middleware-Free Estimators

PRISM-Loc separates *what* a localization node promises its consumers from *how* a pose is estimated. We first state the estimation problem and the output contract shared by all backends, then describe the two-layer architecture that realizes the contract (Section 3.1), the shared particle-filter core (Section 3.2), its two measurement models (Section 3.3), the branch-and-bound relocalizer (Section 3.4), the error-state fusion filter (Section 3.5), and the initialization and diagnostic layer (Section 3.6). Every equation below is transcribed from the C++ sources listed in Section C.

Problem Setup: One Contract, Interchangeable Estimators. Let m be a prior map (an occupancy grid or a point cloud), $\mathbf{u}_{1:t}$ the odometry or inertial inputs, and $\mathbf{z}_{1:t}$ the exteroceptive measurements (2D scans, 3D clouds, GNSS fixes). Each backend $b \in \mathcal{B} = \{\text{laser2d}, \text{ndt3d}, \text{fusion3d}\}$ approximates the Bayes-filter belief over the robot state $\mathbf{x}_t$ (Thrun et al., 2005),

$$\text{bel}_b(\mathbf{x}_t) = \eta p_b(\mathbf{z}_t | \mathbf{x}_t, m) \int p_b(\mathbf{x}_t | \mathbf{u}_t, \mathbf{x}_{t-1}) \text{bel}_b(\mathbf{x}_{t-1}) d\mathbf{x}_{t-1}, \quad (1)$$

with a particle set for the two MCL backends and a Gaussian error state for `fusion3d`. Whatever the representation, a backend reports a point estimate $\hat{\mathbf{T}}_{\text{map},\text{base}}$ with a 6×6 covariance $\hat{\Sigma}_t$ and, so as not to fight the odometry source that owns `odom→base_link`, broadcasts only the correction transform

$$\hat{\mathbf{T}}_{\text{map},\text{odom}} = \hat{\mathbf{T}}_{\text{map},\text{base}} \oplus \mathbf{T}_{\text{odom},\text{base}}^{-1}, \quad (2)$$

Table 1: Per-backend topic contract. All backends additionally subscribe `/initialpose` and the TF `odom→base_link`, and broadcast `map→odom` by Equation (2) with the stamp future-dated by `transform_tolerance` (default 0.1 s).

Backend	Estimator core	Inputs	Outputs
laser2d	MCL + likelihood field (Equation (7))	<code>/scan</code> , <code>/map</code> (OccupancyGrid)	<code>~/pose</code> , <code>~/particle_cloud</code>
ndt3d	MCL + NDT score (Equation (8))	<code>/points</code> , <code>.pcd</code> map file	<code>~/pose</code> , <code>~/particle_cloud</code>
fusion3d	15-state ESKF (Equations (12) and (13))	<code>/points</code> , <code>/imu</code> , <code>/gnss</code> (NavSatFix), <code>.pcd</code> map file	<code>~/pose</code> , <code>~/odometry</code>

where $\oplus$ is pose composition and $\mathbf{T}_{\text{odom},\text{base}}$ is read from TF at the measurement stamp. Equation (2) guarantees the round trip $\hat{\mathbf{T}}_{\text{map},\text{odom}} \oplus \mathbf{T}_{\text{odom},\text{base}} = \hat{\mathbf{T}}_{\text{map},\text{base}}$, which ROS-free unit tests assert to 10^{-9} on $SE(2)$ and $SE(3)$. The design objective is that the tuple (TF edge, pose topic, seeding topic) is identical for all $b \in \mathcal{B}$, so that the sensor suite, not the software stack, selects b .

3.1. Separating Estimators from Middleware: Two Layers, One Contract

PRISM-Loc is organized as two layers realized by four packages (Figure 1). The lower layer holds the estimator cores: `prism_loc_core` implements the MCL family and the relocalizer, and `prism_loc_fusion` implements the ESKF. Both are plain C++17 libraries whose only third-party dependency is Eigen; each `CMakeLists.txt` links `Eigen3::Eigen` alone, with no reference to `roscpp`, `tf2`, or `PCL`. The upper layer holds the ROS 2 node packages `prism_loc` (the `laser2d` and `ndt3d` backends, selected by a backend string parameter) and `prism_loc_fusion_ros` (the `fusion3d` backend). Only these depend on the middleware, and the dependency direction is strictly upward.

The split buys three properties. First, *unit-testability without a running graph*: each core builds and tests with a plain toolchain and GoogleTest, and continuous integration exercises this ROS-free path directly. Second, *determinism*: all sampling routes through a single seeded `std::mt19937_64` wrapper (ROS parameter `seed`, default 42), so a prediction, weighting, or resampling step is reproducible bit for bit, which makes the core tests assertable rather than merely statistical. Third, *one output contract* (Table 1): every backend broadcasts `map→odom` per Equation (2), publishes a `PoseWithCovarianceStamped` on `~/pose`, and accepts `/initialpose`; the MCL nodes add a particle cloud and the fusion node an `Odometry` message. Thin adapters convert `LaserScan`, `OccupancyGrid`, `PointCloud2`, and `.pcd` files into core value types, and in `fusion3d` an adapter wraps PCL’s NDT registration to turn a live cloud into a 6-DoF pose measurement, so PCL appears only inside adapters. QoS profiles follow publisher semantics: sensor streams are subscribed with `SensorDataQoS` (best-effort), while the occupancy grid is subscribed `transient_local` and `reliable` so the single latched map from `map_server` reaches a late-joining node.

3.2. Sampling and Weighting Particles: One MCL Core for 2D and 3D

Both pose-tracking backends share one particle filter over the planar pose $\mathbf{x} = (x, y, \theta)$ (Pose2D), with belief $\{(\mathbf{x}_t^{[i]}, w_t^{[i]})\}_{i=1}^{N_t}$. The recursion is the one wired in `localization_node.cpp` and summarized in Algorithm 1.

Motion Model. `OdometryMotionModel` implements the sampling form of the odometry model

([Thrun et al., 2005](#)). Consecutive odometry poses are decomposed into $(\delta_{r1}, \delta_t, \delta_{r2})$, with $\delta_{r1} = 0$ whenever $\delta_t < 10^{-3}$ m (pure-rotation guard), and each particle is advanced by the noise-corrupted increment

$$\begin{aligned}\hat{\delta}_{r1} &= \delta_{r1} - \varepsilon(\alpha_1 \delta_{r1}^2 + \alpha_2 \delta_t^2), & \hat{\delta}_{r2} &= \delta_{r2} - \varepsilon(\alpha_1 \delta_{r2}^2 + \alpha_2 \delta_t^2), \\ \hat{\delta}_t &= \delta_t - \varepsilon(\alpha_3 \delta_t^2 + \alpha_4 (\delta_{r1}^2 + \delta_{r2}^2)),\end{aligned}\quad (3)$$

where $\varepsilon(\sigma^2)$ draws zero-mean Gaussian noise of variance σ^2 and $\alpha_{1..4}$ are the alpha1–alpha4 parameters.

Weighting. Given a measurement model with log-likelihood $\ell(\mathbf{x})$ (Section 3.3), the correct step of `ParticleFilter` replaces each weight by

$$w^{[i]} = \frac{\exp(\ell(\mathbf{x}^{[i]}) - \ell_{\max})}{\sum_j \exp(\ell(\mathbf{x}^{[j]}) - \ell_{\max})}, \quad \ell_{\max} = \max_j \ell(\mathbf{x}^{[j]}), \quad (4)$$

subtracting the per-step maximum for numerical stability. The weights are overwritten rather than multiplied into the previous ones; this coincides with the standard recursion after every resampling step (which leaves uniform weights), and otherwise discards the previous step’s weights, a simplification we note rather than hide.

KLD-Adaptive Resampling. Resampling is triggered only when the effective sample size $N_{\text{eff}} = (\sum_i w^{[i]})^2 / \sum_i (w^{[i]})^2$ falls below τN (`resample_threshold` $\tau = 0.5$). Samples are then drawn with probability proportional to weight and dropped into a histogram with bins of `kld_bin_xy` in (x, y) and `kld_bin_yaw` in θ . Following [Fox \(2003\)](#), after $k > 1$ distinct bins have been occupied the target sample count is

$$M_k = \frac{k-1}{2 \epsilon_{\text{kld}}} \left(1 - \frac{2}{9(k-1)} + \sqrt{\frac{2}{9(k-1)}} z_{1-\delta} \right)^3, \quad (5)$$

and drawing stops once $n \geq \max(M_k, N_{\min})$ or $n = N_{\max}$, with ϵ_{kld} (`kld_err`), $z_{1-\delta}$ (`kld_z`), and $[N_{\min}, N_{\max}]$ (`min_particles`, `max_particles`) as parameters. The resampled set receives uniform weights.

Pose Estimate. The reported pose is the weighted mean of (x, y) with the circular mean of θ ,

$$\hat{x} = \sum_i w^{[i]} x^{[i]}, \quad \hat{y} = \sum_i w^{[i]} y^{[i]}, \quad \hat{\theta} = \text{atan2}(\sum_i w^{[i]} \sin \theta^{[i]}, \sum_i w^{[i]} \cos \theta^{[i]}), \quad (6)$$

and the 6×6 covariance carries the weighted planar second moments in (x, y) and θ , while z , roll, and pitch are marked unobserved with variance 10^6 .

3.3. Scoring Observations: Likelihood-Field and NDT Measurement Models

A backend differs from the other only through a `MeasurementModel`, whose single virtual method returns the unnormalized log-likelihood $\ell(\mathbf{x})$ of the current observation given a candidate base pose.

2D Likelihood Field. `Laser2DLikelihoodField` is the likelihood-field model ([Thrun et al., 2005](#)). At construction the occupancy grid (cells with value ≥ 50 occupied) is converted into a field $d(\cdot)$ holding the Euclidean distance to the nearest occupied cell, computed exactly by the separable squared-distance transform of [Felzenszwalb and Huttenlocher \(2012\)](#) and clamped at `likelihood_max_dist`. For a scan with ranges r_j at bearings ϕ_j , the model visits every s -th beam, $s = \max(1, \lfloor n/\text{max_beams} \rfloor)$, discards ranges outside $(r_{\min}, r_{\max})$, and scores

$$\ell_{2D}(\mathbf{x}) = \sum_{j \in \mathcal{J}} \log \max \left(z_{\text{hit}} \exp(-d(\mathbf{e}_j(\mathbf{x}))^2 / 2\sigma_{\text{hit}}^2) + z_{\text{rand}} / r_{\max}, 10^{-12} \right), \quad (7)$$

Algorithm 1: One laser2d/ndt3d filter update, as wired in `localization_node.cpp`

Input: particle set $\mathcal{X}$; previous and current odometry poses $\mathbf{o}_{t-1}, \mathbf{o}_t$; measurement model ℓ ;
gates $d_{\min}, a_{\min}$

```

1 if  $\|\mathbf{o}_t - \mathbf{o}_{t-1}\|_{xy} \geq d_{\min}$  or  $|\theta(\mathbf{o}_t) - \theta(\mathbf{o}_{t-1})| \geq a_{\min}$  or a re-seed forced an update then
2   for  $i \leftarrow 1$  to  $|\mathcal{X}|$  do
3      $\mathbf{x}^{[i]} \leftarrow$  sample Equation (3) from  $(\mathbf{o}_{t-1}, \mathbf{o}_t)$  applied to  $\mathbf{x}^{[i]}$ ;
4     compute weights  $w^{[i]}$  by Equation (4);
5     if  $N_{\text{eff}} < \tau|\mathcal{X}|$  then
6        $\mathcal{X} \leftarrow$  KLD-resample until  $n \geq \max(M_k, N_{\min})$  (Equation (5)) or  $n = N_{\max}$ ;
7      $\mathbf{o}_{t-1} \leftarrow \mathbf{o}_t$ ;

```

Output: $(\hat{x}, \hat{y}, \hat{\theta})$ and covariance by Equation (6); $\hat{\mathbf{T}}_{\text{map,odom}}$ by Equation (2)

where $\mathbf{e}_j(\mathbf{x})$ is the beam endpoint $r_j(\cos \phi_j, \sin \phi_j)$ transformed into the map through $\mathbf{x}$ composed with the sensor mounting pose.

3D NDT Score. Ndt3DModel scores a 3D point cloud against a Normal Distributions Transform map (Biber and Straßer, 2003; Magnusson et al., 2007). NdtMap voxelizes the prior cloud at resolution `ndt_resolution`; each voxel v with at least `voxel_min_points` points stores its mean μ_v and the inverse of its covariance after inflating every eigenvalue to at least 10^{-3} of the largest. With $\mathbf{T}(\mathbf{x})$ the 3D lift of the planar particle at the fixed height `base_height` composed with the sensor pose, the per-particle score over a subsampled cloud $\mathcal{P}$ (at most `max_points` points) is

$$\ell_{3D}(\mathbf{x}) = \sum_{\mathbf{p} \in \mathcal{P}} \log(s(\mathbf{T}(\mathbf{x})\mathbf{p}) + s_{\text{floor}}), \quad s(\mathbf{q}) = \exp\left(-\frac{1}{2}(\mathbf{q} - \mu_{v(\mathbf{q})})^\top \Sigma_{v(\mathbf{q})}^{-1}(\mathbf{q} - \mu_{v(\mathbf{q})})\right), \quad (8)$$

with $s(\mathbf{q}) = 0$ when $\mathbf{q}$ falls in an empty voxel and $s_{\text{floor}} = 10^{-6}$. This is the NDT-as-measurement-model (NDT-MCL) idea: the score that Newton registration usually optimizes is read out as a particle likelihood, and the 3D backend estimates the same (x, y, θ) as the 2D one.

3.4. Relocalizing by Branch-and-Bound: Deterministic Kidnapped-Robot Recovery

Recovery from a lost or kidnapped estimate is handled by `BranchAndBoundMatcher`, a correlative scan matcher (Olson, 2009) bounded by a max-pooled pyramid as in Hess et al. (2016). At construction the likelihood field (clamped at $4\sigma_{\text{hit}}$) is converted into a probability grid and a pyramid of $D + 1$ levels (`bbs_max_depth D`),

$$P_0(c) = \exp(-d(c)^2/2\sigma_{\text{hit}}^2), \quad P_h(c_x, c_y) = \max_{0 \leq a, b < 2^h} P_0(c_x + a, c_y + b), \quad (9)$$

where cells outside the grid count as zero; P_h is built by doubling, $P_h(c) = \max\{P_{h-1}(c), P_{h-1}(c + 2^{h-1}\mathbf{e}_x), P_{h-1}(c + 2^{h-1}\mathbf{e}_y), P_{h-1}(c + 2^{h-1}\mathbf{1})\}$. A search at center $\mathbf{c}$ with per-axis half-windows (L_x, L_y) in cells enumerates rotations $\theta_k \in \Theta$ over $\pm \text{bbs_angular_window}$ at `bbs\_angular\_step` and the *symmetric* set of integer cell offsets $\mathcal{O} = \{-L_x, \dots, L_x\} \times \{-L_y, \dots, L_y\}$. In the node's default *map-sized* mode (`bbs_global_window`), for a $w \times h$ grid $\mathbf{c}$ is the center of cell $(\lfloor w/2 \rfloor, \lfloor h/2 \rfloor)$ and $L_x = \max(1, \min(\lfloor w/2 \rfloor, \text{round}(W_{\max}/\text{res})))$, $L_y = \max(1, \min(\lfloor h/2 \rfloor, \text{round}(W_{\max}/\text{res})))$ with the compute cap $W_{\max} = \text{bbs_max_linear_window}$ (50 m). Offset $\mathbf{o}$ then places the robot at the center of cell $(\lfloor w/2 \rfloor, \lfloor h/2 \rfloor) + \mathbf{o}$, so unless the cap binds, the candidate positions are exactly the centers of all grid cells (plus at most one off-grid row or column, which the feasibility rule below removes); when the cap binds, the node logs a warning that relocalization is not global. With

bbs_global_window disabled, $L_x = L_y = \text{round}(W/\text{res})$ about the map midpoint for the linear window $W = \text{bbs_linear_window}$. A candidate $(k, \mathbf{o})$ is *feasible* when the robot position $\mathbf{c} + \text{res} \cdot \mathbf{o}$ falls in an on-grid cell with occupancy below 50, i.e. a free or unknown (-1) cell; the feasible set is $\mathcal{F} \subseteq \Theta \times \mathcal{O}$. (Version 0.1 of the code anchored the roots at $-L, -L + 2^D, \dots \leq L$ without trimming, so it enumerated the asymmetric superset $[-L, -L + 2^D \lceil (2L + 1)/2^D \rceil]^2$, -10 m to $+15.5$ m about the map midpoint for the defaults, and it had no feasibility rule.) For rotation θ_k , let $\mathbf{g}_j^k$ be the grid cell of the j -th subsampled endpoint ($j \in \mathcal{E}$: every s -th beam with a valid range, $s = \max(1, \lfloor n/\text{bbs_max_beams} \rfloor$) as in Equation (7), so $|\mathcal{E}|$ can reach $2 \text{bbs_max_beams} - 1$) at the center translation. A node at depth h with anchor $\mathbf{o}$ covers the offsets $\mathbf{o} + [0, 2^h]^2$ and is scored by

$$S_h(k, \mathbf{o}) = \sum_{j \in \mathcal{E}} P_h(\kappa_h(\mathbf{g}_j^k + \mathbf{o})), \quad (10)$$

where at $h = 0$ out-of-grid cells contribute zero, and at $h \geq 1$ a block that lies entirely outside the grid contributes zero while any other anchor is clamped into the grid by κ_h . Leaves ($h = 0$) carry the exact correlative score S_0 , which is the correlative surrogate of Hess et al. (2016); Olson (2009), not the MCL log-likelihood of Equation (7).

Proposition 1 (Admissible bound). *For every node $(k, \mathbf{o})$ at depth $h \geq 1$ and every leaf offset $\mathbf{o}' \in \mathbf{o} + [0, 2^h]^2$, $S_h(k, \mathbf{o}) \geq S_0(k, \mathbf{o}')$.*

Proof. Fix an endpoint j and let $\mathbf{c} = \mathbf{g}_j^k + \mathbf{o}$. If the block $\mathbf{c} + [0, 2^h]^2$ lies entirely outside the grid, every leaf cell $\mathbf{g}_j^k + \mathbf{o}'$ is also outside and contributes zero, so both sides are 0. Otherwise, when $\mathbf{c}$ is inside the grid $P_h(\mathbf{c})$ is by Equation (9) the maximum of P_0 over the block, which contains the leaf cell. When a coordinate of $\mathbf{c}$ is negative, clamping it to 0 yields a block $[0, 2^h]$ in that coordinate that contains every in-grid cell of $[c, c + 2^h]$, because $c < 0$; out-of-grid leaf cells contribute $0 \leq P_h$. Hence each endpoint's term at depth h upper-bounds its term at every leaf of the node, and summing over j gives the claim. (Without the clamp, a coarse node whose corner falls off the map would under-count and could prune the optimum; the implementation comment in `bbs.cpp` records exactly this case.) $\square$

Algorithm 2 tiles $\mathcal{O}$ with root anchors $-L_x, -L_x + 2^D, \dots \leq L_x$ (and likewise in y), expands nodes depth-first with children visited in decreasing order of their bound, discards any child whose anchor exceeds L_x or L_y , prunes a subtree whenever its bound does not exceed the best feasible leaf score found so far, and skips infeasible leaves.

Corollary 1 (Optimality over the feasible set). *If $\mathcal{E} \neq \emptyset$ and $\mathcal{F} \neq \emptyset$, Algorithm 2 returns a candidate $(k^*, \mathbf{o}^*) \in \mathcal{F}$ with $S_0(k^*, \mathbf{o}^*) = \max_{(k, \mathbf{o}) \in \mathcal{F}} S_0(k, \mathbf{o})$; ties go to the first maximizer reached.*

Proof. *Coverage.* Per axis, with $L \in \{L_x, L_y\}$: for $o \in \{-L, \dots, L\}$ the root anchor $a = -L + 2^D \lfloor (o + L)/2^D \rfloor$ satisfies $a \leq o \leq L$, so o lies in a root block; every ancestor of the leaf o inside that block has an anchor $\leq o \leq L$ and is never discarded by the trimming rule. Trimming therefore removes only blocks that contain no offset of $\mathcal{O}$, and every candidate of $\mathcal{O}$ is a leaf of the enumerated tree; conversely every leaf has anchor $\leq L$ in each axis and anchor $\geq -L$ by construction, so the leaves are exactly $\Theta \times \mathcal{O}$. *Pruning.* The incumbent S^* is only ever set from a feasible leaf. A subtree is pruned only when its bound is $\leq S^*$, and by Proposition 1 (which holds for every leaf of a block, feasible or not, inside $\mathcal{O}$ or not) no leaf below it scores more than S^* . Hence no feasible leaf with a strictly larger score is ever discarded, and the final incumbent attains the maximum over $\mathcal{F}$. (With $\mathcal{E} = \emptyset$, i.e. no valid range, the code returns an invalid result without searching.) $\square$

The guarantee holds for pyramid values stored in single precision and summed in double: each per-endpoint term at depth h is an exact maximum of leaf values, and the endpoints are summed in

Algorithm 2: Branch-and-bound global relocalization (bbs . cpp)

Input: scan endpoints $\mathcal{E}$; pyramid $\{P_h\}_{h=0}^D$; rotations $\{\theta_k\}$; half-windows L_x, L_y ; feasible set $\mathcal{F}$; acceptance fraction ρ

```

1  $C \leftarrow \{(k, \mathbf{o}, D, S_D(k, \mathbf{o})) : \forall k, o_x \in \{-L_x, -L_x + 2^D, \dots\} \leq L_x, o_y \in \{-L_y, -L_y + 2^D, \dots\} \leq L_y\}$ ;
2  $S^* \leftarrow -1$ ;
3 Procedure Branch( $C$ )
4   sort  $C$  by bound, descending;
5   foreach  $(k, \mathbf{o}, h, U) \in C$  do
6     if  $U \leq S^*$  then break // admissible prune;
7     if  $h = 0$  then
8       if  $(k, \mathbf{o}) \in \mathcal{F}$  then  $S^* \leftarrow U$ ;  $\mathbf{x}^* \leftarrow (\mathbf{c} + \text{res} \cdot \mathbf{o}, \theta_k)$ ;
9     else
10      Branch( $\{(k, \mathbf{o} + \delta, h-1, S_{h-1}(k, \mathbf{o} + \delta)) : \delta \in \{0, 2^{h-1}\}^2, o_x + \delta_x \leq L_x, o_y + \delta_y \leq L_y\}$ );
11 Branch( $C$ );
Output:  $\mathbf{x}^*$ , valid iff  $S^* \geq 0$  and  $S^* \geq \rho|\mathcal{E}|$  (Equation (11))

```

the same order at every depth, so monotonicity of rounded addition preserves the inequality. The unit tests check it directly: on randomized small cases the branch-and-bound score equals, bit for bit, that of an exhaustive search over $\mathcal{F}$ (matchExhaustive), and a manual mutation check (not part of the suite) confirms the test’s power: giving coarse nodes the leaf’s drop rule instead of the clamp, the inadmissible bound of the proof’s remark, makes it fail. It is optimality on the rotation–translation lattice only, not over continuous poses, and for the correlative surrogate S_0 , not for the posterior. In map-sized mode it is *global* over the grid’s cell centers whenever the cap $W_{\max}$ does not bind. The match is accepted only when

$$S_0^* \geq \rho|\mathcal{E}|, \quad (11)$$

with $\rho = \text{bbs_min_score_fraction}$ and $|\mathcal{E}|$ the number of valid endpoints used; an accepted pose re-seeds the particle filter with a Gaussian of standard deviation (0.2 m, 0.2 m, 0.1 rad), whereas a rejected one leaves the estimate untouched. The search is deterministic and therefore reproducible in a unit test.

3.5. Fusing IMU, LiDAR, and GNSS: A 15-State Error-State Kalman Filter

The fusion3d backend runs ESKF, which follows the quaternion error-state formulation of Solà (2017). The nominal state carries position $\mathbf{p}$, velocity $\mathbf{v}$, orientation $\mathbf{q}$, and accelerometer and gyroscope biases $\mathbf{b}_a, \mathbf{b}_g$; the error state is $\delta\mathbf{x} = (\delta\mathbf{p}, \delta\mathbf{v}, \delta\theta, \delta\mathbf{b}_a, \delta\mathbf{b}_g) \in \mathbb{R}^{15}$ with a local angular error $\delta\theta$.

Prediction. On each IMU sample $(\mathbf{a}_m, \omega_m)$ with period Δt , let $\mathbf{R} = \mathbf{R}(\mathbf{q})$, $\mathbf{a} = \mathbf{a}_m - \mathbf{b}_a$, $\omega = \omega_m - \mathbf{b}_g$, and $\mathbf{a}_w = \mathbf{R}\mathbf{a} + \mathbf{g}$ with $\mathbf{g} = (0, 0, -9.81)$. The nominal state and error covariance evolve as

$$\mathbf{p} += \mathbf{v}\Delta t + \frac{1}{2}\mathbf{a}_w\Delta t^2, \quad \mathbf{v} += \mathbf{a}_w\Delta t, \quad \mathbf{q} \leftarrow \mathbf{q} \otimes \text{Exp}(\omega\Delta t), \quad \mathbf{P} \leftarrow \mathbf{F}\mathbf{P}\mathbf{F}^\top + \mathbf{Q}, \quad (12)$$

where Exp is the $SO(3)$ exponential, $\mathbf{F}$ is the 15×15 identity except for the blocks

$$\mathbf{F}_{pv} = \mathbf{I}\Delta t, \quad \mathbf{F}_{v\theta} = -\mathbf{R}[\mathbf{a}]_\times \Delta t, \quad \mathbf{F}_{vb_a} = -\mathbf{R}\Delta t, \quad \mathbf{F}_{\theta\theta} = \mathbf{R}\{\text{Exp}(\omega\Delta t)\}^\top, \quad \mathbf{F}_{\theta b_g} = -\mathbf{I}\Delta t,$$

and the process noise

$$\mathbf{Q} = \text{blkdiag}(\mathbf{0}_3, \sigma_a^2 \Delta t^2 \mathbf{I}_3, \sigma_g^2 \Delta t^2 \mathbf{I}_3, \sigma_{b_a}^2 \Delta t \mathbf{I}_3, \sigma_{b_g}^2 \Delta t \mathbf{I}_3)$$

is built from `sigma_acc`, `sigma_gyro`, `sigma_acc_bias`, and `sigma_gyro_bias`; $\mathbf{R}\{\cdot\}$ denotes the rotation matrix of a quaternion.

Correction, Injection, Reset. Two updates share the Kalman gain,

$$\mathbf{K} = \mathbf{P}\mathbf{H}^\top (\mathbf{H}\mathbf{P}\mathbf{H}^\top + \mathbf{R}_z)^{-1}, \quad \delta\hat{\mathbf{x}} = \mathbf{K}\mathbf{y}, \quad \mathbf{P} \leftarrow (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}. \quad (13)$$

`updatePose` applies a 6-DoF pose from NDT registration with $\mathbf{H}$ selecting $(\delta\mathbf{p}, \delta\theta)$ and residual $\mathbf{y} = (\mathbf{p}_m - \mathbf{p}, \text{Log}(\mathbf{q}^{-1} \otimes \mathbf{q}_m))$, with $\mathbf{R}_z = \text{diag}(\sigma_{\text{pos}}^2 \mathbf{I}, \sigma_{\text{rot}}^2 \mathbf{I})$ (`pose_pos_std`, `pose_rot_std`); the node applies it only when PCL’s NDT converged with fitness at most `ndt_max_fitness`. `updatePosition` applies a 3-DoF GNSS fix, converted from WGS-84 through ECEF to a local ENU frame about a datum (the first accepted fix unless `use_datum` fixes one), with $\mathbf{H}$ selecting $\delta\mathbf{p}$ and $\mathbf{R}_z$ taken from the fix covariance (horizontal variances floored at 10^{-4} , vertical at 1). After each update the error is injected, additively for $\mathbf{p}$, $\mathbf{v}$, $\mathbf{b}_a$, $\mathbf{b}_g$ and as $\mathbf{q} \leftarrow \mathbf{q} \otimes \text{Exp}(\delta\hat{\theta})$ for orientation, and the error mean is reset to zero. The implementation uses the simple covariance update of Equation (13). By default it omits the reset Jacobian (it takes $\mathbf{G} = \mathbf{I}$), a common first-order simplification of the full formulation in Solà (2017); the option `reset_jacobian` applies $\mathbf{P} \leftarrow \mathbf{G}\mathbf{P}\mathbf{G}^\top$ with $\mathbf{G} = \text{blkdiag}(\mathbf{I}, \mathbf{I}, \mathbf{I} - [\frac{1}{2}\delta\hat{\theta}]_\times, \mathbf{I}, \mathbf{I})$ after each injection, and Section 4.3 measures its effect on consistency.

3.6. Initializing and Diagnosing: Making Every Precondition Observable

All backends accept a `PoseWithCovarianceStamped` on `/initialpose`: the MCL backends spread a Gaussian particle cloud at the supplied mean and covariance, and the fusion backend loads it as its nominal state. Automatically, `laser2d` runs Algorithm 2 before its first pose when `try_global_localization` is set, or on demand through the `~/global_localization` service (the matcher is constructed lazily at the first call). `fusion3d` auto-initializes once it holds both an attitude, roll and pitch from the first IMU specific force $\mathbf{f}$ and heading ψ_0 from `initial_yaw`,

$$\phi_0 = \text{atan2}(f_y, f_z), \quad \vartheta_0 = \text{atan2}(-f_x, \sqrt{f_y^2 + f_z^2}), \quad \mathbf{q}_0 = \mathbf{R}_z(\psi_0) \mathbf{R}_y(\vartheta_0) \mathbf{R}_x(\phi_0), \quad (14)$$

and a position from the first GNSS fix that passes the gate $\{\text{status} \geq s_{\min}\} \wedge \{\text{lat, lon not NaN}\} \wedge \{c_{xx} \leq c_{\max}\}$ (`gnss_min_status`, `gnss_max_pos_cov`).

Because a localization node that silently does nothing is the dominant first-run frustration, four diagnostic mechanisms accompany the estimators: (i) `ndt3d` and `fusion3d` abort construction with an actionable error when `map_pcd_path` is empty or fails to load; (ii) every callback that cannot yet act emits a rate-limited warning naming the unmet precondition and the remedy; (iii) a 10 s startup watchdog reports each silent mandatory input together with its live publisher count, separating “no publisher” from “publisher present but QoS-incompatible”; and (iv) the GNSS gate and the NDT update log each rejection with the measured value against the violated threshold. Every tunable is a declared ROS 2 parameter documented one-to-one in `PARAMS.md` (Section B). These mechanisms carry no algorithmic weight; they exist so that a misconfiguration is diagnosable from the console log alone, in the spirit of the logging infrastructure Nav2 relied on in its long-duration field experiment (Macenski et al., 2020).

4. Experiments

We validate PRISM-Loc at the level of *correctness and closed-loop behavior*, not field accuracy. Section 4.1 describes the test suite and the five synthetic closed-loop experiments, Section 4.2 reports their results, and Section 4.3 isolates the contribution of individual mechanisms using the same committed runs. All data in this section are synthetic, and we make no accuracy claim about physical robots.

4.1. Experimental Setup

Correctness Tests. The suite comprises 52 GoogleTest cases across the four packages: 31 in `prism_loc_core` (both MCL backends and global recovery), 15 in `prism_loc_fusion` (the ESKF), 4 in the `prism_loc` adapter layer, and 2 in `prism_loc_fusion_ros`. The motion model is checked deterministically (zero-command and pure-translation cases) and statistically (the mean of noisy samples approximates the commanded motion); both measurement models are checked for the monotonicity a likelihood must satisfy (the true pose out-scores a displaced one); resampling, the weighted-mean estimate, the grid and NDT-map primitives, and branch-and-bound recovery of a single-scan kidnapped pose (with rejection of an empty scan) are tested directly, the latter also on a 40 m×30 m map with the pose 17 m from the center, together with the symmetric window, the compute cap, the rejection of occupied and off-map candidates, and bit-exact agreement with exhaustive search (Corollary 1); two integration tests close the `laser2d` and `ndt3d` loops until convergence to ground truth. The ESKF is pinned for rest under a static IMU, constant-acceleration kinematics, measurement pull-in, and tracking of a moving target from a wrong initial guess; $SO(3)$ exponential/logarithm round trips, geodetic ENU round trips, and the Equation (2) composition are verified separately, and RNG tests assert that equal seeds reproduce identical sequences; two tests pin the optional ESKF reset Jacobian. Two CI jobs guard the boundary from both sides: a *core* job builds the two cores with plain CMake and runs `ctest` with only a compiler and Eigen, and a ROS job runs `colcon build` and `colcon test` for all four packages in a `ros:humble` container.

Synthetic Closed-Loop Protocol. Five drivers under `experiments/` compile the *actual* core sources directly with g++ and Eigen (no ROS, no CMake) and generate their own worlds, scans, and inertial streams, with all randomness derived from fixed compile-time seeds. The sensor-noise models match the filters' assumptions by construction, so the experiments demonstrate behavior rather than benchmark accuracy.

- *MCL tracking.* A 12 m × 10 m grid at 0.1 m/cell (room border plus two interior pillars) hosts a robot driven around a closed oval loop with semi-axes 3.6 m and 2.6 m for 65 s at 10 Hz (650 steps). Each step ray-casts a 180-beam scan at the true pose and synthesizes drifting odometry from noise-corrupted increments under the `laser2d.yaml` model ($\alpha_{1..4} = 0.2$). The filter, with every parameter from `laser2d.yaml`, is seeded with a broad Gaussian prior ($\sigma = 1$ m, 0.4 rad) at the true start and updated as in Algorithm 1 with $d_{\min} = a_{\min} = 0.2$. Seeds: 101, 202, 303.
- *BBS relocation.* On an asymmetric 6 m × 6 m room (two interior wall stubs break the four-fold symmetry, the world of the BBS unit test), $N = 100$ poses are drawn uniformly over free space with uniform yaw, and one 180-beam scan is ray-cast per pose. To span easy to hard cases, each beam is replaced with a uniform-random spurious range with a per-query probability drawn uniformly in $[0, 0.8]$, modeling people, glass, and objects absent from the map. The real matcher runs with the `bbs_*` defaults ($W = 10$ m, $\pm\pi$ at 0.0175 rad $\approx 1^\circ$, $D = 6$, `bbs_max_beams` = 120, which for a 180-beam scan gives stride 1, so all 180 beams are scored), centered on the room, so its window covers the whole map. The committed CSV was produced by the first release of the matcher; re-running the driver against the current matcher (symmetric window, feasibility rule) reproduces every column except wall-clock latency. Seed: 20240607.
- *ESKF under aiding dropout.* The real `Eskf` runs over a 60 s 3D trajectory, a horizontal figure-eight $x = 8 \sin \omega t$, $y = 4 \sin 2\omega t$ with $\omega = 2\pi/30$ and a ± 1.5 m altitude oscillation. The analytic specific force and angular rate are corrupted with exactly the white-noise densities and constant biases the process model assumes (`fusion3d.yaml` defaults), plus fixed true biases the filter must estimate from zero. The IMU runs at 200 Hz, a simulated NDT source delivers 6-DoF poses at 10 Hz and a simulated GNSS source positions at 1 Hz, with noise matched to the filter's $\mathbf{R}_z$;

Table 2: laser2d steady-state tracking error on synthetic data ($t \geq 10$ s, 110 logged samples per seed at 2 Hz). Columns are computed from `data/mcl_tracking.csv`; the last two rows pool the three seeds as mean $\pm$ std of the per-seed values and as statistics over all 330 samples.

Seed	Mean pos. err. (m)	Max pos. err. (m)	Mean yaw err. (rad)	Max yaw err. (rad)
101	0.113	0.269	0.034	0.120
202	0.100	0.257	0.030	0.105
303	0.102	0.288	0.033	0.184
Per-seed mean $\pm$ std	0.105 $\pm$ 0.007	–	0.032 $\pm$ 0.002	–
Pooled (330 samples)	0.105	0.29	0.032	0.18

both aiding streams are withheld for $t \in [25, 35]$ s. Seed: 20240517.

- *BBS on large maps.* Square warehouse-like grids of side 20, 40, and 80 m at 0.1 m/cell carry border walls, *repeated* shelf rows (0.5 m deep at a 6 m pitch with 1–2 random gaps each) and random 0.4 m pillars; the repetition deliberately creates look-alike places. Kidnapped poses are drawn uniformly over free space and stratified into 60 inside and 60 outside the set that the first release searched (-10 m to $+15.5$ m about the map center; the 20 m map lies entirely inside it). One 180-beam, 30 m-range scan per pose receives the room study’s clutter model, and the first-release matcher (a verbatim copy, centered on the map midpoint with $W = 10$ m) and the current map-sized matcher process the identical scan. Seed: 20260928.
- *ESKF Monte Carlo NEES.* $N = 100$ runs of the ESKF scenario above in which the initial position, velocity, and attitude errors and the true IMU biases are drawn from the filter’s prior $\mathcal{N}(\mathbf{0}, \mathbf{P}_0)$, and the true biases then random-walk with the filter’s bias process noise, so that the prior, the sensor and bias noise, and the aiding noise all match the filter’s model; the one unmodeled effect is that the IMU samples an analytic trajectory, whose integration between samples the zero-order-hold process model does not represent exactly. Every 0.5 s we average the normalized estimation error squared $\epsilon = \mathbf{e}^\top \mathbf{P}^{-1} \mathbf{e}$ over the runs, for the full 15-D error and for each 3-D block, and compare it with the two-sided 95% interval $[\chi_{0.025}^2(Nn)/N, \chi_{0.975}^2(Nn)/N]$ for n degrees of freedom (Bar-Shalom et al., 2001). The shipped filter and the reset-Jacobian variant process identical data. No parameter was tuned on these runs. Seeds: 20260929 plus the run index, and 20261029 plus the run index for the bias random walk.

Baselines, Metrics, and Hardware. No external baseline (e.g., `nav2_amcl` or `hdl_localization`) is run: the synthetic worlds are designed to exercise PRISM-Loc’s own cores, and a comparative accuracy study on public data is left to future work (Section 5). We report position error (Euclidean, m) and absolute yaw error against synthetic ground truth, BBS success rate (position error < 0.5 m and yaw error $< 10^\circ$), acceptance and false-accept counts at the threshold ρ of Equation (11), windowed RMSE, the 3σ envelope, and Monte Carlo NEES for the ESKF, and BBS wall-clock latency. All accuracy numbers are deterministic; latency is machine-dependent and was measured single-threaded on the machine that produced the committed CSV.

4.2. Main Results

MCL Tracking and KLD Adaptation. Figure 2 and Table 2 show that after convergence ($t \geq 10$ s, across all three seeds) the `laser2d` filter holds a mean position error of 0.105 m (max 0.29 m) and a mean yaw error of 0.032 rad, i.e. 1.8° (max 0.18 rad), with little spread across seeds (0.105 $\pm$ 0.007 m over per-seed means). The KLD bound (Equation (5)) sizes the set to the posterior: it starts at the 2000-particle ceiling under the broad prior, collapses to the 500-particle floor after the first

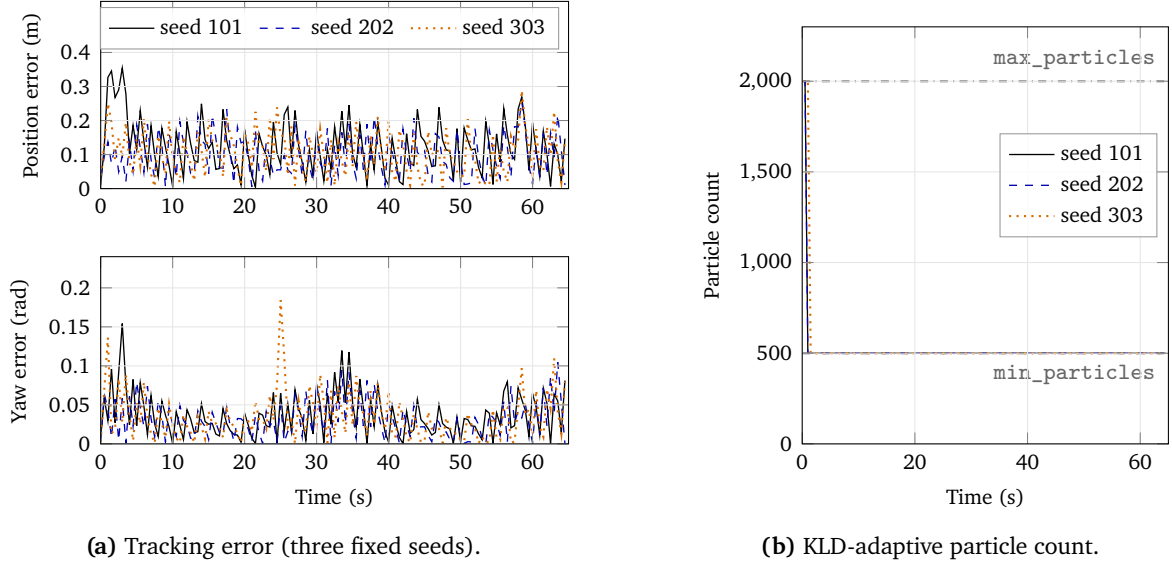


Figure 2: laser2d closed-loop tracking on synthetic data. A simulated robot drives a closed oval loop (semi-axes 3.6 m and 2.6 m) for 65 s at 10 Hz in a 12 m×10 m grid with two symmetry-breaking pillars; the real ParticleFilter with Laser2DLikelihoodField and KLD resampling is updated exactly as in Algorithm 1. (a) Starting from a broad prior, the filter converges within a few updates; across the three seeds the steady-state ($t \geq 10$ s) mean position error is 0.105 m (max 0.29 m) and the mean yaw error 0.032 rad (max 0.18 rad). (b) The particle set starts at the 2000-particle ceiling and collapses to the 500-particle floor after the first post-convergence update, where it stays.

post-convergence update, and holds there for the rest of the loop, a 4× reduction in per-update likelihood evaluations once the posterior is concentrated.

Global Relocalization. Of the 100 kidnapped-robot queries, 97 succeed (Table 3); over all queries the recovered pose has median position error 0.037 m (95th percentile 0.072 m) and median yaw error 0.48° (95th percentile 1.74°), consistent with the 0.1 m grid and $\approx 1^\circ$ rotation lattice. Figure 3 shows that the score fraction of Equation (11) is an informative confidence signal: the three failures score in 0.21–0.27 with more than 3 m and roughly 90° – 180° error, all below $\rho = 0.40$, so the default threshold yields *zero false accepts* while accepting 78 matches, all correct, and conservatively rejecting 19 correct fixes. The median score fraction is 0.632 and successes span 0.22–0.97, so three low-scoring successes overlap the failure band. Median full-map query latency is 37.65 ms.

ESKF Fusion. With full aiding the windowed position RMSE is 0.016 m (Table 3). During the 10 s blackout the filter dead-reckons on the IMU alone and the RMSE rises to 0.125 m, with a peak error of 0.256 m at $t = 34.5$ s at the end of the blackout; the first aiding cycle brings the error back to 0.015 m by $t = 35$ s, and the post-dropout RMSE is 0.009 m (Figure 4). The filter’s own 3σ bound inflates from roughly 0.03–0.04 m under full aiding to 0.663 m and stays above the true error at every logged sample, i.e. the reported position covariance is not overconfident on this single seeded run. This is a one-sided envelope check on one run; the NEES study below is the consistency test.

Relocalization on Large Maps. Table 4 moves the kidnapped-robot study to maps larger than the room. For poses outside the set the first release searched, that matcher succeeds on only 2 of 60 queries on the 40 m map and 0 of 60 on the 80 m map, and at $\rho = 0.40$ it *accepts* a wrong in-window pose on 39 and 51 of them; the map-sized matcher recovers 50 and 41 of the same 60 poses. Inside that set the two matchers perform alike, as they should. The map-sized window does not, however, make acceptance safe. At $\rho = 0.40$ the current matcher accepts a wrong pose on 6 of 120 queries on the 40 m map and 15 of 120 on the 80 m map (first release: 43 and 57). Of these 21 false accepts, 13

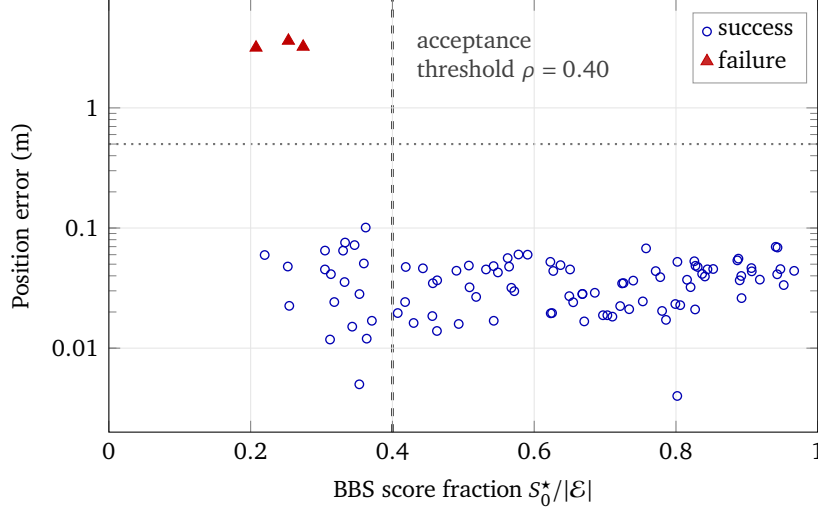


Figure 3: Branch-and-bound relocalization of a kidnapped robot on synthetic data. Each marker is one of $N = 100$ queries in an asymmetric 6 m×6 m room: position error versus the score fraction of Equation (11). Each 180-beam scan is corrupted with a per-query clutter probability drawn uniformly in $[0, 0.8]$. Success means position error < 0.5 m (dotted) and yaw error $< 10^\circ$. All three failures score below the default threshold (dashed), so every accepted match is correct (zero false accepts); 19 correct fixes are conservatively rejected.

are 180° flips and none is closer than 4.95 m to the truth; they are look-alike aisles, not near misses. No threshold removes them cheaply: the highest-scoring false match on the two large maps has score fraction 0.71, and only 94 of the 198 successes score above it. The feasibility rule does act: the first release returned a pose inside an obstacle or off the map on 8 of its 300 queries, the current matcher on none. Global coverage also costs time: the median query takes 29 ms on the 20 m map but 1526 ms on the 80 m map, with a 95th percentile of 6374 ms under heavy clutter.

ESKF Consistency. The NEES test gives a negative verdict (Table 5, Figure 5): the filter is overconfident. Its time-averaged full-state NEES is 18.05 against 15 degrees of freedom, and at single epochs the run average lies inside the 95% interval at only 31 of 120 epochs and above it (overconfident) at the other 89; it is never below it. The blocks locate the problem. Attitude is the worst block (above at 47 epochs, peak 5.91 against an upper limit of 3.50); position (inside at 104 of 120 epochs) and velocity are overconfident mainly during and just after the aiding dropout; both bias blocks are consistent (inside at 116 and 120 of 120 epochs). The published orientation covariance is therefore too small, and the position covariance is too small while aiding is absent. A first version of this study held the true biases constant while the filter injects bias random-walk noise; with that mismatch both bias blocks read as underconfident, and with the bias random walk added to the truth they are consistent.

4.3. Ablation Studies

The committed runs also isolate the effect of four mechanisms. All numbers below are recomputed from the same CSVs; no additional experiment was run for this section.

Acceptance Threshold. Table 6 sweeps ρ over the recorded score fractions. Without the gate ($\rho \leq 0.20$) all three failures would re-seed the filter at a wrong pose; any $\rho \in [0.30, 0.60]$ removes every false accept, and the shipped $\rho = 0.40$ trades 19 rejected correct fixes for a margin of 0.13 above the highest-scoring failure. Because a rejected match merely waits for the next scan, whereas a false accept silently corrupts the posterior, we keep the conservative default; $\rho = 0.30$ would have accepted 16 more correct fixes on this data.

Table 3: Relocalization and fusion results on synthetic data. Left: branch-and-bound over $N = 100$ kidnapped-robot queries (data/bbs_relocalization.csv); percentiles use linear interpolation; latency is single-threaded wall-clock on the machine that produced the committed CSV. Right: ESKF position error by window relative to the 10 s aiding dropout (data/eskf_track.csv, logged at 2 Hz).

BBS relocalization	
Success (pos. < 0.5 m, yaw $< 10^\circ$)	97 / 100
Median / p95 position error	0.037 / 0.072 m
Median / p95 yaw error	0.48° / 1.74°
Accepted at $\rho = 0.40$ / false accepts	78 / 0
Median score fraction	0.632
Median latency (min, p95)	37.65 ms (8.77, 106.07)
ESKF fusion	
RMSE before dropout ($1 \leq t < 25$ s)	0.016 m
RMSE during dropout ($25 \leq t < 35$ s)	0.125 m
RMSE after dropout ($t \geq 35$ s)	0.009 m
Peak error (at $t = 34.5$ s)	0.256 m
Peak 3σ bound (at $t = 34.5$ s)	0.663 m
Samples with error $> 3\sigma$	0 / 121

Table 4: Relocalization on large synthetic maps: v0.1 window versus map-sized window. Warehouse-like maps with repeated shelf rows at 0.1 m/cell; kidnapped poses stratified by whether they lie inside (in) or outside (out) the set the v0.1 node searched (-10 m to $+15.5$ m about the map center); the 20 m map lies entirely inside it. Each cell reads v0.1 / current matcher on identical scans ($U[0, 0.8]$ beam clutter). Success: position error < 0.5 m and yaw error $< 10^\circ$; accepted: $S_0^* \geq 0.40|\mathcal{E}|$; false accept: accepted but not a success. Latency is the single-threaded median (data/bbs_largemap.csv, generated by experiments/bbs_largemap.cpp).

Map	Poses	N	Success	Accepted	False accepts	Median latency (ms)
20 m	in	60	60 / 60	50 / 50	0 / 0	29 / 29
40 m	in	60	53 / 54	57 / 57	4 / 3	74 / 127
40 m	out	60	2 / 50	41 / 48	39 / 3	150 / 220
80 m	in	60	51 / 53	54 / 55	6 / 4	391 / 1547
80 m	out	60	0 / 41	51 / 52	51 / 11	334 / 1484

Clutter. Table 7 shows how clutter affects the matcher. Below a clutter probability of 0.533 every query succeeds and is accepted; in the top third the pose is still recovered in 30 of 33 cases with an unchanged median error, but the median score fraction falls to 0.353, so only 11 are accepted. Clutter therefore degrades the matcher’s *confidence* much faster than its *accuracy*, and the gate absorbs the difference; all three failures occur in this top third.

KLD Adaptation and Aiding. Replacing KLD adaptation with a fixed set at the 2000-particle ceiling would cost four times the steady-state likelihood evaluations of the 500-particle floor; averaged over each whole run, including the initial transient, the adaptive set holds 526.9 particles, $3.8\times$ fewer than the ceiling, with the accuracy of Table 2. The dropout window acts as an ablation of both aiding streams: removing NDT and GNSS updates raises position RMSE from 0.016 m to 0.125 m over 10 s, and restoring them recovers 0.009 m, so the growth is the bounded drift of IMU-only integration from which the filter recovers within one aiding cycle.

Reset Jacobian. Enabling reset_jacobian on identical data changes the run-averaged NEES of Table 5 by at most 0.026 at any epoch and in any block, and leaves every in/above/below count unchanged, so the omitted reset Jacobian is not the source of the inconsistency; with 10 Hz pose

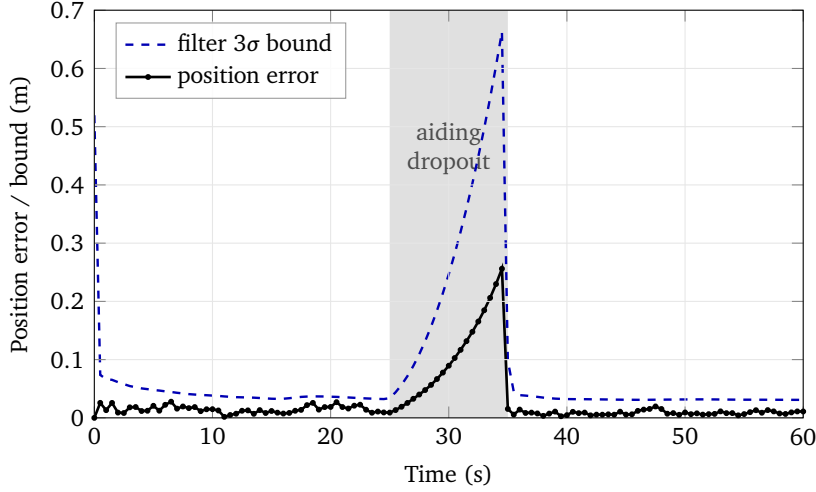


Figure 4: ESKF fusion under a 10 s aiding dropout on synthetic data. A 200 Hz IMU is fused with 10 Hz NDT-like 6-DoF pose and 1 Hz GNSS-like position updates over a 60 s 3D figure-eight; both aiding streams are withheld for $t \in [25, 35]$ s (shaded). The position error grows to its peak of 0.256 m on dead reckoning alone while the filter’s own 3σ bound inflates to 0.663 m, and both collapse within one aiding cycle once updates resume. The bound stays above the true error at every logged sample.

Table 5: Monte Carlo NEES of the ESKF ($N = 100$ runs, 120 epochs at 2 Hz after $t = 0$; initial error and true IMU biases drawn from $\mathbf{P}_0$). Band: two-sided 95% interval for the run-averaged NEES; a consistent filter keeps about 95% of epochs inside it. In / above / below count epochs for the shipped filter ($\mathbf{G} = \mathbf{I}$); the last column is the time-averaged NEES with the reset Jacobian enabled on identical data (data/eskf_nees.csv, experiments/eskf_nees.cpp).

Error block (dof)	95% band	Mean NEES, $\mathbf{G} = \mathbf{I}$	In / above / below	Mean NEES, reset $\mathbf{G}$
Full state (15)	[13.95, 16.09]	18.05	31 / 89 / 0	18.04
Position $\delta \mathbf{p}$ (3)	[2.54, 3.50]	3.10	104 / 15 / 1	3.10
Velocity $\delta \mathbf{v}$ (3)	[2.54, 3.50]	3.15	94 / 25 / 1	3.15
Attitude $\delta \boldsymbol{\theta}$ (3)	[2.54, 3.50]	3.48	73 / 47 / 0	3.48
Accel. bias $\delta \mathbf{b}_a$ (3)	[2.54, 3.50]	3.09	116 / 4 / 0	3.09
Gyro bias $\delta \mathbf{b}_g$ (3)	[2.54, 3.50]	3.12	120 / 0 / 0	3.12

updates the injected attitude corrections are small, which is consistent with $\mathbf{G} \approx \mathbf{I}$. We have not isolated the cause; candidates are the zero-order-hold discretization of the synthetic IMU, which the process model does not account for, and the first-order treatment of the attitude–bias coupling.

5. Discussion and Limitations

What the Factoring Buys. Because the MCL core is written against one `MeasurementModel` interface, the 2D and 3D backends share the motion model, weighting, KLD resampling, estimate, and every test of those components; adding a backend means adding one likelihood function. Because the cores are middleware-free, every equation of Section 3 is exercised by deterministic tests and by the closed-loop drivers of Section 4 without a ROS installation, which is also what makes the evaluation reproducible bit for bit from the repository.

Cost Analysis. The per-update cost of the MCL backends is linear in the particle count and in the number of scored beams or points (Equations (7) and (8)); KLD adaptation keeps the former at the

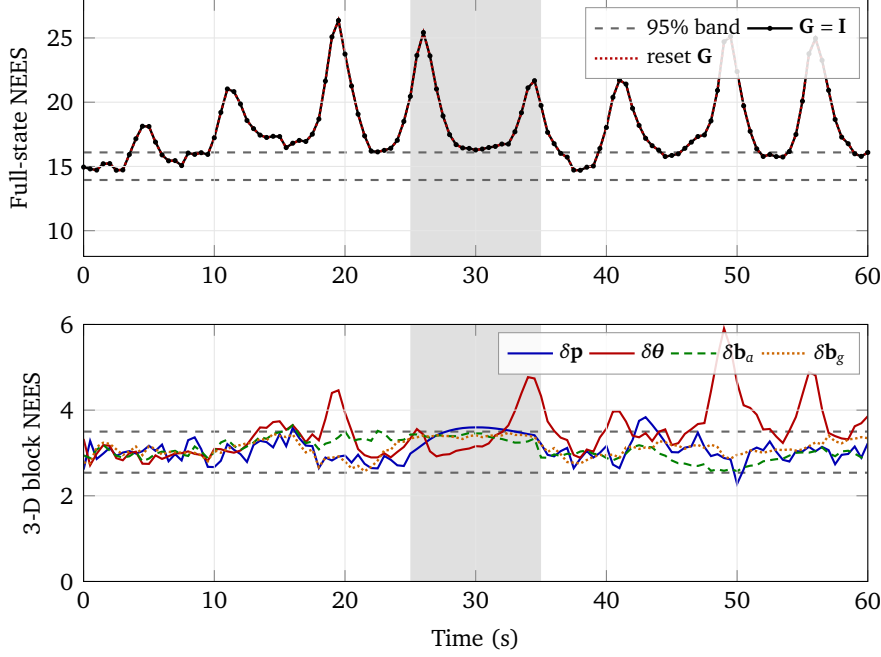


Figure 5: Monte Carlo NEES of the ESKF on synthetic data ($N = 100$ runs of the Figure 4 scenario with initial errors and IMU biases drawn from $\mathbf{P}_0$; aiding dropout shaded). Top: run-averaged NEES of the full 15-D error state for the shipped filter ($\mathbf{G} = \mathbf{I}$) and with the reset Jacobian, which overlap; dashed lines bound the two-sided 95% interval. Bottom: 3-D blocks of the shipped filter against the 3-dof interval. The full-state NEES lies above the interval at most epochs and never below it; attitude is the most overconfident block, position and velocity exceed the interval mainly during and after the dropout, and both bias blocks stay inside it.

Table 6: Ablation of the BBS acceptance threshold ρ (Equation (11)), recomputed post hoc from the score fractions in `data/bbs_relocalization.csv`; no search is re-run. “Rejected correct” counts successful matches that the threshold would discard. The shipped default is $\rho = 0.40$.

ρ	0.20	0.25	0.30	0.35	0.40	0.45	0.50	0.60
Accepted	100	98	94	84	78	73	67	54
False accepts	3	2	0	0	0	0	0	0
Rejected correct	0	1	3	13	19	24	30	43

500-particle floor once the posterior is concentrated (Figure 2). The branch-and-bound matcher’s cost is data-dependent through pruning: on the synthetic room a full-map query took a median of 37.65 ms (min 8.77 ms, 95th percentile 106.07 ms) on the machine that produced the committed data, and it runs only when relocalization is requested, not every scan. Because the map-sized window grows with map area, so does the cost: on the 80 m map of Table 4 the median query takes 1526 ms, which is acceptable for an on-demand recovery but not for per-scan use; `bbs_max_linear_window` bounds it at the price of global coverage. The ESKF step is a fixed 15×15 covariance propagation per IMU sample.

Limitations. PRISM-Loc is an architecture paper, and its scope is deliberately bounded. (i) *Planar state in the MCL backends.* Both particle-filter backends estimate (x, y, θ) ; `ndt3d` evaluates a full 3D Gaussian-voxel map from that planar pose at a fixed base height, which suits wheeled robots on locally flat floors but does not track roll, pitch, or vertical motion. Full $SE(3)$ state exists only in `fusion3d`. (ii) *2D-only relocalization with an imperfect acceptance test.* Algorithm 2 operates on a 2D occupancy pyramid; `ndt3d` and `fusion3d` rely on an operator-supplied or GNSS-derived initial

Table 7: Ablation by clutter level. The 100 BBS queries of Figure 3 split into thirds of the clutter range $[0, 0.8]$ by the per-query `clutter_frac` column; success as in Table 3, acceptance at $\rho = 0.40$.

Clutter probability	Queries	Success	Accepted	Median pos. err. (m)	Median score fraction
$[0, 0.267)$	37	37	37	0.040	0.831
$[0.267, 0.533)$	30	30	30	0.032	0.624
$[0.533, 0.8]$	33	30	11	0.037	0.353

pose and local tracking. The search window is sized from the map and restricted to non-occupied cells, but the only guard against a wrong optimum is the score fraction of Equation (11), and on maps with repeated structure it is not enough: at the default ρ the matcher accepted a wrong pose on 6 of 120 (40 m) and 15 of 120 (80 m) synthetic queries, and no single threshold removes these on the two maps together without rejecting more than half of the correct fixes. An accepted match replaces the particle set at once, and relocalization runs only at startup or on a service call; no loss detector triggers it. Verifying the top candidates, e.g., by running MCL on several hypotheses before committing, would address this and is not implemented. Beyond the compute cap (100 m per axis by default) the search is no longer global, which the node reports at startup. (iii) *No time-offset or extrinsic estimation.* The ESKF treats the LiDAR/GNSS-to-IMU time offset and the LiDAR-IMU extrinsic as known inputs. (iv) *Loosely-coupled LiDAR update.* In `fusion3d`, PCL NDT produces a 6-DoF pose measurement that the ESKF fuses as a loosely-coupled update, which keeps the filter core independent of the registration engine but forgoes the accuracy of tightly-coupled formulations such as FAST-LIO (Xu and Zhang, 2021). The filter is also not NEES-consistent on our synthetic scenario (Table 5): it is overconfident, most of all in attitude and, while aiding is absent, in position and velocity, whereas its bias blocks are consistent; the optional reset Jacobian of Solà (2017) does not change this. Consumers should treat the published orientation covariance, and the position covariance during aiding outages, with caution until the cause is found. (v) *Single static map.* All backends assume one prior map of a static world, loaded once, with no online map update or change detection. (vi) *No field-accuracy evaluation.* The evaluation of Section 4 is deterministic unit and integration testing plus a seeded synthetic study whose worlds and sensor-noise models match the filters’ assumptions by construction, and it runs no external baseline. It establishes that each core behaves as specified in closed loop, not how accurate PRISM-Loc is on a physical robot. We have not measured trajectory error on physical or public datasets; the planned evaluation will report absolute and relative trajectory error (ATE/RPE) following the TUM RGB-D conventions (Sturm et al., 2012) on the per-backend `map→odom` estimates. Until then we make no accuracy claim.

Future Work. Extending the MCL core to a 6-DoF particle state or coupling it to an external attitude source, hypothesis verification for the 2D relocalizer and a 3D branch-and-bound or place-recognition front end for the 3D backends, a diagnosis of the ESKF’s attitude overconfidence, joint time-offset and extrinsic estimation in the ESKF, and the public-dataset evaluation above are the next steps on the roadmap.

6. Conclusion

We presented PRISM-Loc, a LiDAR localization stack whose contribution is architectural rather than algorithmic: three known estimator families, likelihood-field MCL, NDT-MCL, and an error-state Kalman filter fusing LiDAR, IMU, and RTK-GNSS, sit behind a single ROS 2 topic and TF contract, so that a 2D LiDAR, a 3D LiDAR, or a sensor-fused configuration is a deployment choice rather than a separate software stack. Each estimator is a pure C++ core with no dependence on the

middleware runtime, the two MCL backends share one predict–weight–resample core that differs only in its measurement model, and `laser2d` relocalizes deterministically by branch-and-bound over a map-sized window of non-occupied cells. Seeded synthetic experiments show that the assembled pipelines behave as intended in closed loop, and they also expose two open problems that we report rather than hide: on large maps with repeated structure the relocalizer’s acceptance test still admits wrong poses, and the ESKF covariance is overconfident, mainly in attitude. A field-accuracy evaluation remains future work. PRISM-Loc is released under the Apache-2.0 license at https://github.com/kjungmo/prism_loc.

References

- Y. Bar-Shalom, X. R. Li, and T. Kirubarajan. *Estimation with Applications to Tracking and Navigation: Theory, Algorithms and Software*. John Wiley & Sons, New York, 2001. doi: 10.1002/0471221279.
- P. Biber and W. Straßer. The normal distributions transform: A new approach to laser scan matching. In *Proceedings of the 2003 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, volume 3, pages 2743–2748, Las Vegas, NV, USA, Oct. 2003. IEEE. doi: 10.1109/IROS.2003.1249285.
- F. Dellaert, D. Fox, W. Burgard, and S. Thrun. Monte Carlo localization for mobile robots. In *Proceedings of the 1999 IEEE International Conference on Robotics and Automation (ICRA)*, volume 2, pages 1322–1328, Detroit, MI, USA, May 1999. IEEE. doi: 10.1109/ROBOT.1999.772544.
- P. F. Felzenszwalb and D. P. Huttenlocher. Distance transforms of sampled functions. *Theory of Computing*, 8(19):415–428, 2012. doi: 10.4086/toc.2012.v008a019.
- D. Fox. Adapting the sample size in particle filters through KLD-sampling. *The International Journal of Robotics Research*, 22(12):985–1003, 2003. doi: 10.1177/0278364903022012001.
- D. Fox, W. Burgard, F. Dellaert, and S. Thrun. Monte Carlo localization: Efficient position estimation for mobile robots. In *Proceedings of the Sixteenth National Conference on Artificial Intelligence (AAAI/IAAI)*, pages 343–349, Orlando, FL, USA, 1999. AAAI Press / The MIT Press.
- W. Hess, D. Kohler, H. Rapp, and D. Andor. Real-time loop closure in 2D LIDAR SLAM. In *2016 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1271–1278, Stockholm, Sweden, 2016. IEEE. doi: 10.1109/ICRA.2016.7487258.
- K. Koide, J. Miura, and E. Menegatti. A portable three-dimensional LiDAR-based system for long-term and wide-area people behavior measurement. *International Journal of Advanced Robotic Systems*, 16(2), Mar. 2019. ISSN 1729-8814. doi: 10.1177/1729881419841532.
- S. Macenski, F. Martín, R. White, and J. G. Clavero. The Marathon 2: A navigation system. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2718–2725. IEEE, 2020. doi: 10.1109/IROS45743.2020.9341207.
- S. Macenski, T. Moore, D. V. Lu, A. Merzlyakov, and M. Ferguson. From the desks of ROS maintainers: A survey of modern & capable mobile robotics algorithms in the Robot Operating System 2. *Robotics and Autonomous Systems*, 168:104493, 2023. doi: 10.1016/j.robot.2023.104493.
- M. Magnusson, A. Lilienthal, and T. Duckett. Scan registration for autonomous mining vehicles using 3D-NDT. *Journal of Field Robotics*, 24(10):803–827, 2007. ISSN 1556-4959. doi: 10.1002/rob.20204.

- T. Moore and D. Stouch. A generalized extended Kalman filter implementation for the robot operating system. In *Intelligent Autonomous Systems 13: Proceedings of the 13th International Conference (IAS-13)*, volume 302 of *Advances in Intelligent Systems and Computing*, pages 335–348, Cham, 2016. Springer. ISBN 978-3-319-08338-4. doi: 10.1007/978-3-319-08338-4_25.
- E. B. Olson. Real-time correlative scan matching. In *2009 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4387–4393, Kobe, Japan, 2009. IEEE. doi: 10.1109/ROBOT.2009.5152375.
- J. Solà. Quaternion kinematics for the error-state Kalman filter, Nov. 2017. URL <https://arxiv.org/abs/1711.02508>. arXiv:1711.02508 [cs.RO], tutorial/technical report.
- J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers. A benchmark for the evaluation of RGB-D SLAM systems. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 573–580, Vilamoura, Portugal, Oct. 2012. IEEE. doi: 10.1109/IROS.2012.6385773.
- S. Thrun, D. Fox, W. Burgard, and F. Dellaert. Robust Monte Carlo localization for mobile robots. *Artificial Intelligence*, 128(1-2):99–141, May 2001. ISSN 0004-3702. doi: 10.1016/S0004-3702(01)00069-8.
- S. Thrun, W. Burgard, and D. Fox. *Probabilistic Robotics*. MIT Press, Cambridge, MA, USA, 2005. ISBN 978-0-262-20162-9.
- G. Wan, X. Yang, R. Cai, H. Li, Y. Zhou, H. Wang, and S. Song. Robust and precise vehicle localization based on multi-sensor fusion in diverse city scenes. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4670–4677, Brisbane, QLD, Australia, May 2018. IEEE. doi: 10.1109/ICRA.2018.8461224.
- W. Xu and F. Zhang. FAST-LIO: A fast, robust LiDAR-inertial odometry package by tightly-coupled iterated Kalman filter. *IEEE Robotics and Automation Letters*, 6(2):3317–3324, Apr. 2021. doi: 10.1109/LRA.2021.3064227.

A. Reproduction

Every number and figure of Section 4 regenerates from the repository alone. The drivers and the core test suites need only a C++17 compiler and the Eigen 3 headers (Debian/Ubuntu packages `g++`, `libeigen3-dev`); the unit tests additionally use GoogleTest (`libgtest-dev`). ROS 2 Humble is required only for the two node packages, which continuous integration builds in a `ros:humble` container (`.github/workflows/ci.yml`). From the repository root:

```
CORE="-I prism_loc_core/include -I prism_loc_core/test -I /usr/include/eigen3"
g++ -O2 -std=c++17 $CORE prism_loc_core/src/*.cpp \
    experiments/mcl_tracking.cpp -o /tmp/exp_mcl && /tmp/exp_mcl
g++ -O2 -std=c++17 $CORE prism_loc_core/src/*.cpp \
    experiments/bbs_relocalization.cpp -o /tmp/exp_bbs && /tmp/exp_bbs
g++ -O2 -std=c++17 -I prism_loc_fusion/include -I /usr/include/eigen3 \
    prism_loc_fusion/src/*.cpp experiments/eskf_fusion.cpp -o /tmp/exp_eskf
/tmp/exp_eskf > docs/paper/data/eskf_track.csv
g++ -O2 -std=c++17 $CORE -I experiments prism_loc_core/src/*.cpp \
    experiments/legacy/bbs_v01.cpp experiments/bbs_largemap.cpp \
    -o /tmp/exp_bbs_large && /tmp/exp_bbs_large # about 7 min
g++ -O2 -std=c++17 -I prism_loc_fusion/include -I /usr/include/eigen3 \
    prism_loc_fusion/src/*.cpp experiments/eskf_nees.cpp -o /tmp/exp_nees
/tmp/exp_nees
python3 docs/paper/make_study_tables.py # tables of the two new studies
python3 docs/paper/check_numbers.py # recompute every quoted statistic
```

All randomness derives from fixed compile-time seeds (Table 8), so every CSV column is bit-for-bit reproducible except the wall-clock `time_ms` latency columns of the two BBS experiments. We re-ran all three drivers from the sources of this commit while preparing this version: `mcl_tracking.csv`, `mcl_particles.csv`, and `eskf_track.csv` were byte-identical to the committed files, and `bbs_relocalization.csv` matched in every column except `time_ms`. On that re-run machine (AMD Ryzen 5 5500GT, single thread) the median BBS query latency was 10.26 ms against the 37.65 ms of the committed CSV, which illustrates why latency is reported as machine-dependent; the re-run CSV is committed under `docs/paper/data/rerun_2026-09-28/`. After the matcher gained its symmetric window and feasibility rule we re-ran the room study once more against the current code (`bbs_relocalization_fixbbs.csv` in the same directory): again every column except `time_ms` is identical to the committed CSV, with a median latency of 9.90 ms on the same re-run machine. The guard script `docs/paper/check_numbers.py` recomputes each headline statistic from the committed CSVs and fails if a number quoted in the $\text{\LaTeX}$ sources, including this package, no longer matches.

Table 8: Experiment drivers, fixed seeds, and committed artifacts. Drivers live under `experiments/`; CSVs under `docs/paper/data/` (copied to `data/` in this package).

Driver	Seed(s)	Figures / tables	CSV(s)
<code>mcl_tracking.cpp</code>	101, 202, 303	Figure 2, Table 2	<code>mcl_tracking.csv</code> , <code>mcl_particles.csv</code>
<code>bbs_relocalization.cpp</code>	20240607	Figure 3, Tables 3, 6 and 7	<code>bbs_relocalization.csv</code>
<code>eskf_fusion.cpp</code>	20240517	Figure 4, Table 3	<code>eskf_track.csv</code>
<code>bbs_largemap.cpp</code>	20260928	Table 4	<code>bbs_largemap.csv</code>
<code>eskf_nees.cpp</code>	20260929 + k	Table 5, Figure 5	<code>eskf_nees.csv</code>

B. Parameters

Table 9 lists the defaults used throughout the paper; each is a declared ROS 2 parameter documented one-to-one in `PARAMS.md`, with deployed overrides in `prism_loc/params/laser2d.yaml`, `prism_loc/params/ndt3d.yaml`, and `prism_loc_fusion_ros/params/fusion3d.yaml`.

Table 9: Default parameters (from the shipped YAML files; symbols as in Section 3).

Parameter	Symbol	Default	Used in
<code>min_particles / max_particles</code>	$N_{\min} / N_{\max}$	500 / 2000	Equation (5)
<code>resample_threshold</code>	τ	0.5	N_{eff} trigger
<code>kld_err / kld_z</code>	$\epsilon_{\text{kld}} / z_{1-\delta}$	0.05 / 2.33	Equation (5)
<code>kld_bin_xy / kld_bin_yaw</code>	–	0.5 m / 0.17 rad	Equation (5)
<code>update_min_d / update_min_a</code>	$d_{\min} / a_{\min}$	0.2 m / 0.2 rad	Algorithm 1
<code>alpha1-alpha4</code>	$\alpha_{1..4}$	0.2	Equation (3)
<code>max_beams</code>	–	60	Equation (7)
<code>z_hit / z_rand / sigma_hit</code>	$z_{\text{hit}} / z_{\text{rand}} / \sigma_{\text{hit}}$	0.5 / 0.5 / 0.2 m	Equations (7) and (9)
<code>likelihood_max_dist</code>	–	2.0 m	Equation (7)
<code>ndt_resolution / voxel_min_points</code>	–	1.0 m / 5	Equation (8)
<code>max_points / base_height</code>	–	500 / 0.0 m	Equation (8)
<code>bbs_global_window</code>	–	true	Section 3.4
<code>bbs_max_linear_window</code>	$W_{\max}$	50.0 m	Section 3.4
<code>bbs_linear_window (if not global)</code>	W	10.0 m	Section 3.4
<code>bbs_angular_window / bbs_angular_step</code>	–	$\pi / 0.0175$ rad	Algorithm 2
<code>bbs_max_depth / bbs_max_beams</code>	$D / -$	6 / 120	Equations (9) and (10)
<code>bbs_min_score_fraction</code>	ρ	0.4	Equation (11)
<code>sigma_acc / sigma_gyro</code>	σ_a / σ_g	0.01 / 0.001	Equation (12)
<code>sigma_acc_bias / sigma_gyro_bias</code>	$\sigma_{b_a} / \sigma_{b_g}$	$10^{-4} / 10^{-5}$	Equation (12)
<code>reset_jacobian</code>	$\mathbf{G}$	false ($\mathbf{G} = \mathbf{I}$)	Section 3.5
<code>pose_pos_std / pose_rot_std</code>	$\sigma_{\text{pos}} / \sigma_{\text{rot}}$	0.1 m / 0.05 rad	Equation (13)
<code>ndt_max_fitness</code>	–	2.0	NDT gate
<code>gnss_min_status / gnss_max_pos_cov</code>	$s_{\min} / c_{\max}$	0 / 25.0 m ²	GNSS gate
<code>transform_tolerance / seed</code>	–	0.1 s / 42	Equation (2) / RNG

C. Code Map

Table 10 maps each equation and algorithm to the source file that implements it, and Table 11 collects the notation.

Table 10: Where each equation lives. Paths are relative to the repository root; `core/` abbreviates `prism_loc_core/src/`, `fusion/` abbreviates `prism_loc_fusion/src/`, and `node/` and `fnode/` abbreviate `prism_loc/src/localization_node.cpp` and `prism_loc_fusion_ros/src/fusion_localization_node.cpp`.

Equation / algorithm	Source
Equation (2) (map→odom)	<code>node/</code> , <code>fnode/</code> ; tests <code>prism_loc/test/test_tf_math.cpp</code> , <code>prism_loc_fusion_ros/test/test_tf_compose.cpp</code>
Equation (3) (motion model)	<code>core/motion_model.cpp</code>
Equations (4) to (6) (filter)	<code>core/particle_filter.cpp</code>
Algorithm 1 (update order, gates)	<code>node/</code> (<code>runUpdate</code> , <code>onScan</code>)
Equation (7) (likelihood field)	<code>core/measurement_model.cpp</code> , <code>core/occupancy_grid.cpp</code>
Equation (8) (NDT score)	<code>core/measurement_model.cpp</code> , <code>core/ndt_map.cpp</code>
Equations (9) to (11), Algorithm 2	<code>core/bbs.cpp</code> ; re-seeding in <code>node/</code> (<code>onScan</code>)
Equations (12) and (13) (ESKF)	<code>fusion/eskf.cpp</code> , <code>fusion/so3.cpp</code> , <code>fusion/geodetic.cpp</code>
Equation (14), GNSS and NDT gates	<code>fnode/</code> (<code>attitudeFromAccel</code> , <code>onGnss</code> , <code>onPoints</code>)
Corollary 1 check (exhaustive search)	<code>core/bbs.cpp</code> (<code>matchExhaustive</code>); test <code>prism_loc_core/test/test_bbs.cpp</code>
Synthetic drivers	<code>experiments/mcl_tracking.cpp</code> , <code>bbs_relocalization.cpp</code> , <code>eskf_fusion.cpp</code> , <code>bbs_largemap.cpp</code> (with <code>legacy/bbs_v01.cpp</code>), <code>eskf_nees.cpp</code>

Table 11: Notation.

Symbol	Meaning	Symbol	Meaning
$\mathcal{B}$	set of backends	$\mathbf{T}_{a,b}$	pose of frame b in frame a
$\mathbf{x} = (x, y, \theta)$	planar particle pose	$w^{[i]}$	particle weight
$\ell(\mathbf{x})$	measurement log-likelihood	N_{eff}	effective sample size
M_k	KLD sample bound, k bins	$d(\cdot)$	distance to nearest obstacle
$\boldsymbol{\mu}_v, \boldsymbol{\Sigma}_v$	NDT voxel mean, covariance	P_h	BBS pyramid level h
$S_h(k, \mathbf{o})$	BBS node score	$\mathcal{E}$	subsamped scan endpoints
ρ	BBS acceptance fraction	D	BBS pyramid depth
O, L_x, L_y	BBS offset set, half-windows	$\mathcal{F}$	feasible BBS candidates
W_{max}	BBS half-window cap	ϵ	NEES $\mathbf{e}^\top \mathbf{P}^{-1} \mathbf{e}$
$\mathbf{p}, \mathbf{v}, \mathbf{q}$	ESKF position, velocity, attitude	$\mathbf{b}_a, \mathbf{b}_g$	IMU biases
$\delta \mathbf{x} \in \mathbb{R}^{15}$	ESKF error state	$\mathbf{P}$	error covariance
$\mathbf{F}, \mathbf{Q}$	transition, process noise	$\mathbf{H}, \mathbf{R}_z, \mathbf{K}$	update Jacobian, noise, gain
$\mathbf{G}$	ESKF reset Jacobian		